

ChargeHubBerlin - Project Documentation

Team 6

<https://github.com/SamuskaAksumas/heatmap-charging-stations.git>

Muhammed Korkot [919492] Shoaib Ur Rehman Khan [110569] Chirayu Jain [110379]
Montasir Hasan Chowdhury [111836] Leonie Zeller [108386]

1. Introduction and Use Cases

ChargeHubBerlin is an interactive platform providing Berliners with a user-friendly way to find charging stations, check demand by postal code, and contribute to improving charging infrastructure through suggestions.

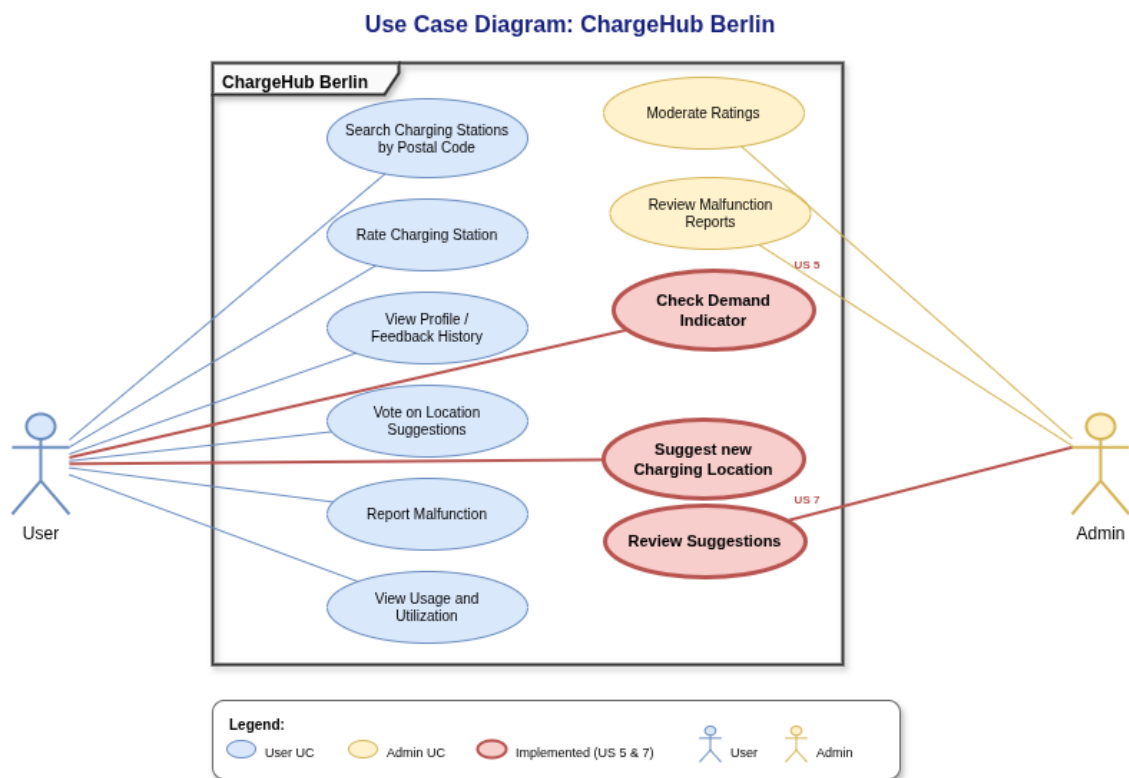


Figure 1: Use Case Diagram showing implemented features (US5 and US7 highlighted)

Table 1: Core Features

Feature	Description
Demand Indicator (US5)	Visualizes charging demand per postal code based on population and existing stations. Formula: $demand = residents \div stations$; residents if no stations exist
Location Suggestions (US7)	Users submit proposed charging locations linked to postal codes. Suggestions are visible in-app and subject to admin moderation.

Table 2: Technology Stack

Category	Technologies
Development	Python 3.10, VS Code, PyCharm
Frontend	Streamlit, Folium
Data Processing	Pandas, GeoPandas
Testing	pytest, pytest-cov, pylint
LLM Support	Claude Code, GitHub Copilot
Collaboration	GitHub, Discord, Miro, Draw.io

2. Domain Modeling and Bounded Contexts

Our project follows Domain-Driven Design with three bounded contexts: **Shared**, **Demand**, and **Suggestion** (see Figure 2).

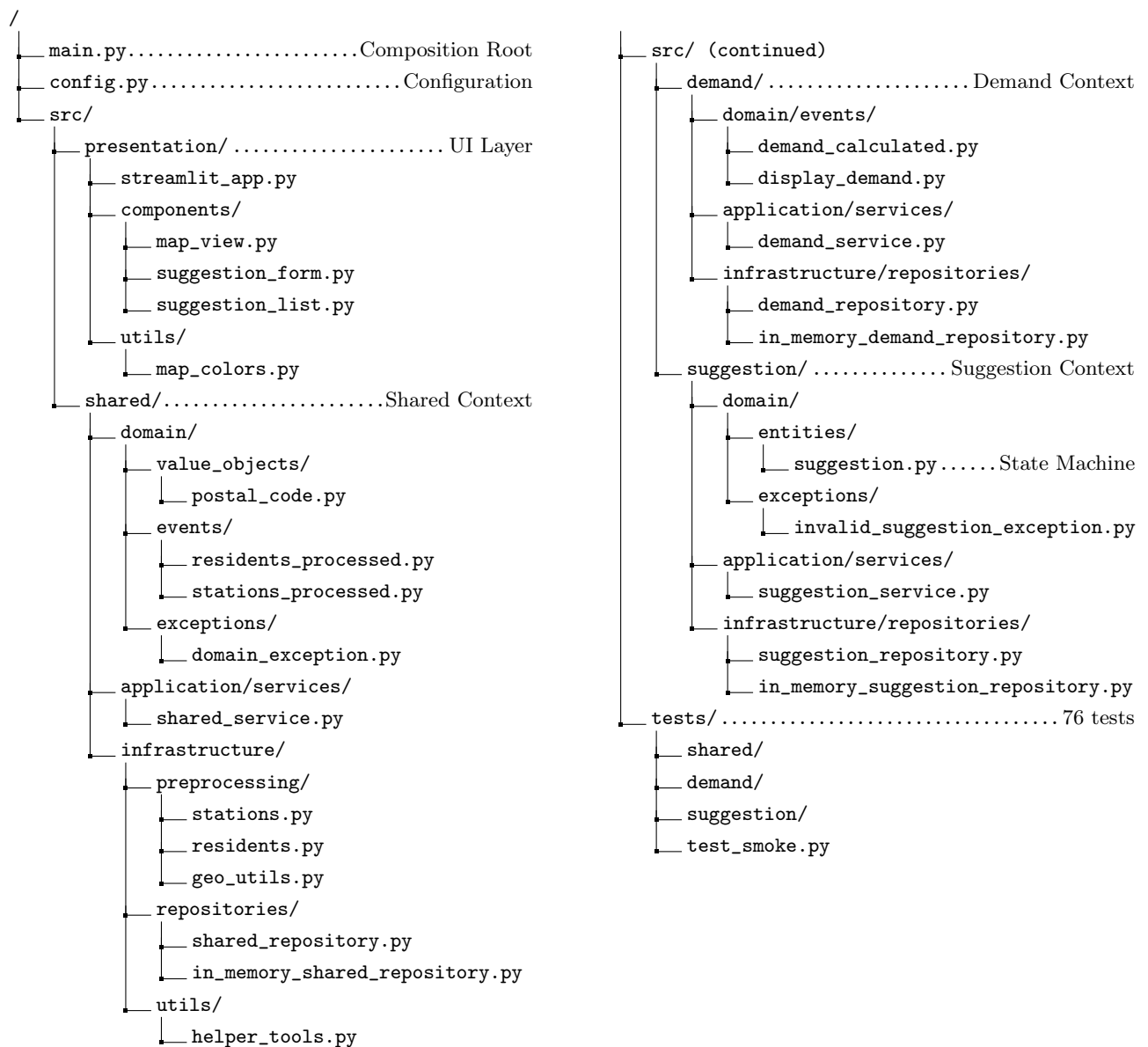


Figure 2: Complete project structure following DDD bounded contexts

2.1 Aggregate Boundaries

Table 3: Aggregate Roots and Boundaries

Aggregate Root	Purpose	Boundary
ChargingSuggestion	Manages suggestion lifecycle with state machine	Per suggestion
DemandResult	Represents demand calculation output	Per request

The **ChargingSuggestion** aggregate encapsulates postal code, address, reason, status, and review meta-data. Domain invariants are enforced through a state machine that allows only valid status transitions.

2.2 Value Object: PostalCode

```

1 @dataclass(frozen=True) # Immutable - cannot be changed after creation
2 class PostalCode:
3     value: str
4
5     def __post_init__(self):
6         if len(self.value) != 5 or not self.value.isdigit():
7             raise InvalidPostalCodeException("Must be 5 digits")
8         if not (10000 <= int(self.value) <= 14200):
9             raise InvalidPostalCodeException("Must be Berlin PLZ")

```

Listing 1: PostalCode Value Object with self-validation

Properties: Immutable (`frozen=True`), Self-validating, Equality by value.

2.3 Domain Event Flows

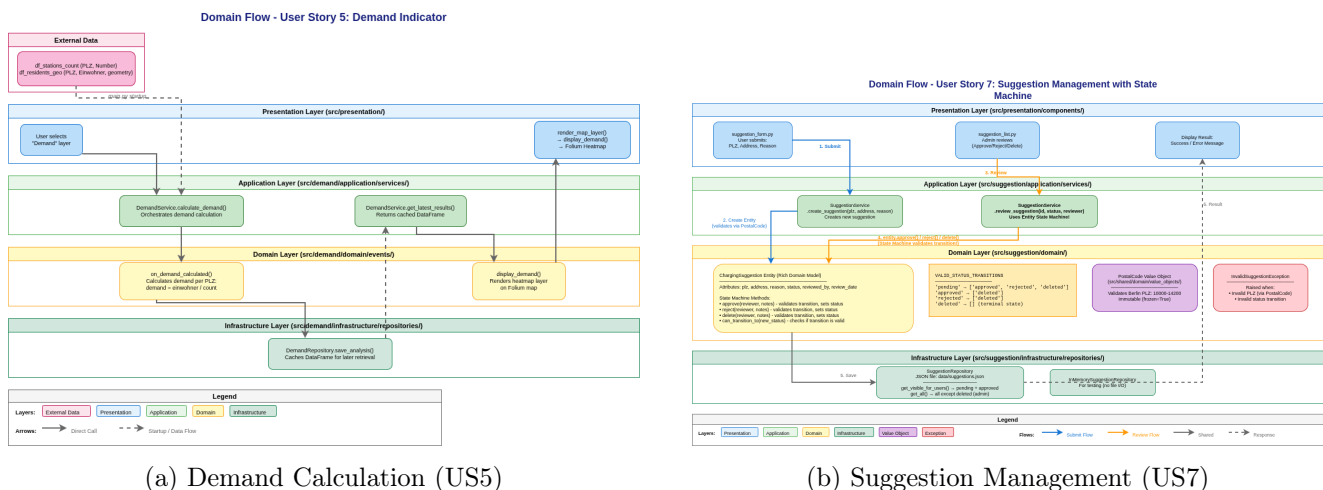


Figure 3: Domain Event Flows for both User Stories

2.4 Sequence Diagrams

User Story 5 – Demand Indicator:

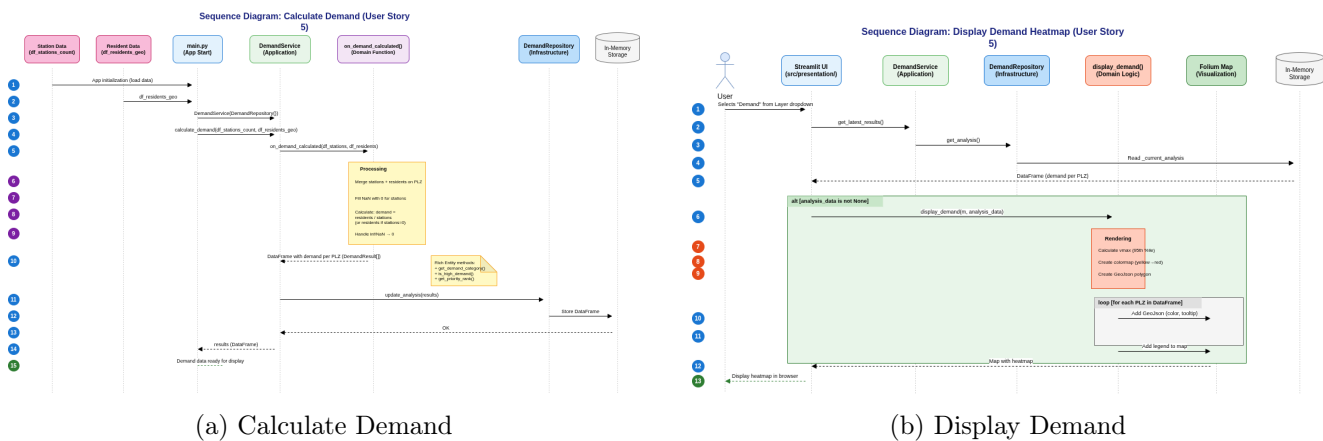


Figure 4: Sequence Diagrams for User Story 5: Demand Indicator

User Story 7 – Location Suggestions:

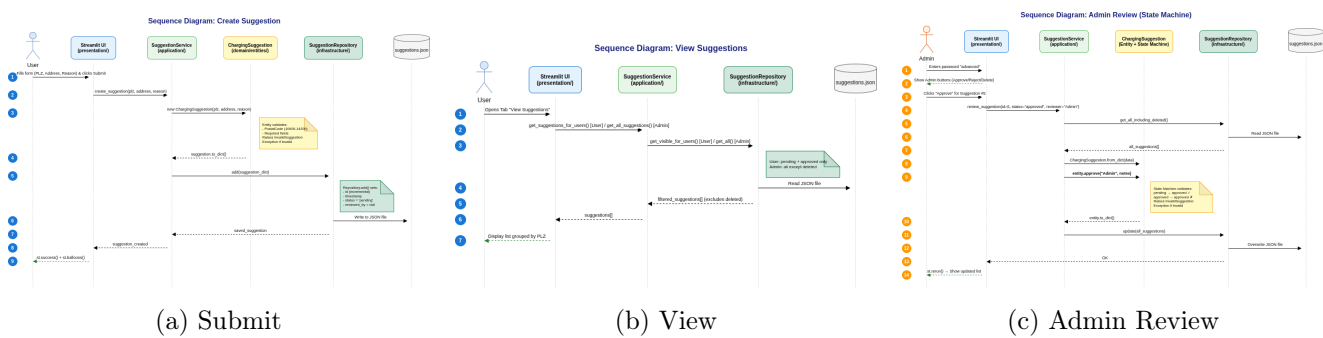


Figure 5: Sequence Diagrams for User Story 7: Location Suggestions

Flow: UI → Application Service → Domain → Infrastructure.

2.5 SOLID Principles Applied

Table 4: SOLID Principles in the Project

Principle	Application
Single Responsibility	Each service has one purpose (SuggestionService, DemandService)
Open/Closed	State Machine – new statuses added without modifying transition logic
Liskov Substitution	InMemorySuggestionRepository replaces production repo in tests
Interface Segregation	Services depend only on repository methods they use
Dependency Inversion	Services depend on abstraction, not implementation

```

1 class SuggestionService:
2     def __init__(self, repository): # Injected - can be real or in-memory
3         self._repository = repository
4
5 # Production: SuggestionService(SuggestionRepository()) -> JSON file
6 # Testing:    SuggestionService(InMemorySuggestionRepository()) -> In-memory

```

Listing 2: Repository Pattern with Dependency Injection

3. Test-Driven Development (TDD)

Our project followed a progressive TDD approach with varying maturity across phases.

Initial Development (Code-First): TDD was not strictly followed at the beginning. The focus was on building a working Streamlit proof-of-concept and understanding data structures.

Tests After Implementation: After initial development, test suites were added for the Demand and Suggestion contexts to cover existing functionality and support refactoring.

True TDD for New Features: For later features, TDD was applied consistently using the Red–Green–Refactor cycle.

Were we always able to adhere to TDD? No. The approach was pragmatic:

- **Code-first:** Initial prototyping and exploration
- **Test-after:** Refactoring and restructuring
- **TDD:** New business-critical features

TDD applied for:

- Admin review functionality (Approve/Reject/Delete)
- State Machine for suggestion status transitions
- User visibility filtering
- PostalCode validation (Value Object)

Test-After used for:

- Preprocessing logic (adopted from original code)
- Streamlit UI components (difficult to unit test)
- Data integration (already validated in Part 1)

3.1 TDD Example: User Visibility Filtering

RED – Write failing test first:

```
1 def test_get_suggestions_for_users_excludes_rejected(self, setup):
2     service, repo = setup
3     service.create_suggestion("10115", "Address", "Reason")
4     service.review_suggestion(1, status="rejected", reviewer="Admin")
5
6     user_visible = service.get_suggestions_for_users() # NOT IMPLEMENTED
7
8     assert 'rejected' not in [s['status'] for s in user_visible]
```

Listing 3: Failing test (RED phase)

Result: `AttributeError: 'SuggestionService' has no attribute 'get_suggestions_for_users'`

GREEN – Implement minimal code:

```
1 def get_suggestions_for_users(self) -> List[Dict]:
2     return [s for s in self._repository.get_all()
3             if s.get('status') in ['pending', 'approved']]
```

Listing 4: Minimal implementation (GREEN phase)

Result: 20 passed in 0.02s

3.2 Test Coverage & Code Quality

We measure code quality using `pytest-cov` for test coverage and `pylint` for static code analysis. A custom `code_quality.py` script aggregates these metrics into a dashboard (Figure 6).

```

=====
CODE QUALITY DASHBOARD
=====

Lines of Code:    1506 lines in 41 files

Running tests with coverage...
-----
TEST COVERAGE
-----
Total:            [#####.....] 62%
Tests Passed:     74

Coverage per Layer (DDD):
Domain            [#####.....] 84% OK
Application       [#####.....] 100% OK
Infrastructure    [#####.....] 82% OK
Presentation      [###.....] 19%

Running pylint...
-----
PYLINT SCORE
-----
Score:            [#####.] 9.74/10
Errors:           0
Warnings:         0
Refactor hints:   0

Pylint checks:
- Code style (PEP8 naming, line length)
- Errors (undefined variables, bad imports)
- Refactoring (duplicate code, complexity)
- Design (too many arguments, deep nesting)

=====
SUMMARY
=====
Domain Layer:     WELL TESTED
Application Layer: WELL TESTED
Code Quality:     GOOD

Business logic is well tested - Presentation/Infrastructure
have lower coverage which is acceptable for a DDD demo.

```

Figure 6: Code Quality Dashboard output from `code_quality.py`

Test Coverage (62% total, 76 tests): The Domain layer (84%) and Application layer (100%) are thoroughly tested using In-Memory Repositories, enabling fast and isolated unit tests without file I/O dependencies. Infrastructure coverage (82%) is slightly lower because some file I/O edge cases are tested via integration rather than unit tests. Presentation layer coverage (19%) is intentionally low – Streamlit UI components are difficult to unit test without mocking the entire framework, so coverage comes primarily from smoke tests verifying imports work correctly. **Pylint Score (9.7/10):** Pylint measures code style (PEP8), errors, complexity, and design issues. We disabled missing docstring warnings (C0114-C0116) since not every helper function requires documentation. The remaining minor issues are duplicate initialization code in the repositories, which we accepted for clarity over abstraction. A score of 8.0 or higher is considered production-ready; our 9.7 indicates high code quality.

3.3 Backend Architecture

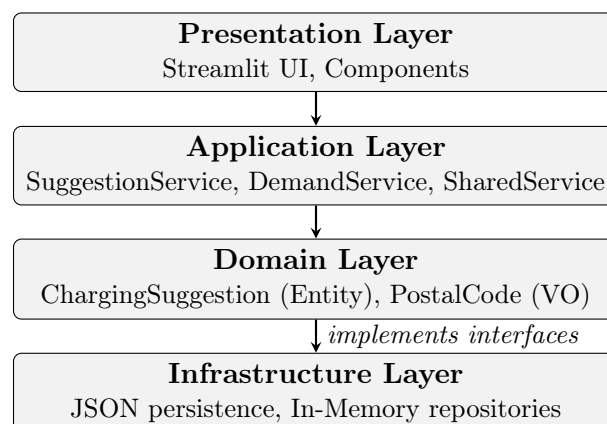


Figure 7: Layered Architecture with Dependency Inversion

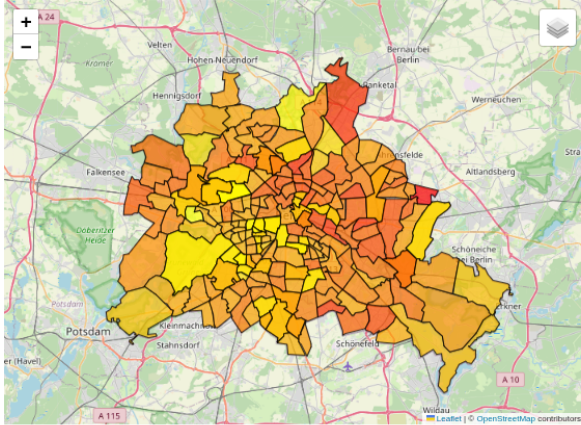
3.4 UI Components

ChargeHub Berlin

Map View Suggest Location View Suggestions

Select Layer

- ☒ Residents
- ☐ Charging_Stations
- ☐ Demand



(a) Demand Heatmap

ChargeHub Berlin

Map View Suggest Location View Suggestions

Suggest New Charging Location

Help improve Berlin's charging infrastructure by suggesting new locations where charging stations are needed.

Postal Code (PLZ) Address/Location Description

Press Enter to submit form

Why is this location needed?

Please enter a valid 5-digit postal code

(b) Suggestion Form with PLZ Validation Error

ChargeHub Berlin

Map View Suggest Location View Suggestions

Community Suggestions

See suggestions from the community for new charging locations.

Enter Admin Password to review

Enter the correct admin password to unlock review features.

Total suggestions: 10

- PLZ 10439 (8 suggestions)
- PLZ 12045 (1 suggestions)
 - [OK] Location: Korkstraße 12
 - Reason: bad
 - Status: Approved
 - Suggested on 2025-12-22 15:06
 - Reviewed by Admin on 2025-12-23
- PLZ 12345 (1 suggestions)

(c) Suggestion List (User View)

Community Suggestions

See suggestions from the community for new charging locations.

Enter Admin Password to review

Admin mode unlocked

Total suggestions: 14

- PLZ 10000 (1 suggestions)
- PLZ 10001 (1 suggestions)
- PLZ 10439 (8 suggestions)
- PLZ 12045 (2 suggestions)
- PLZ 12345 (2 suggestions)
 - Location: 123123
 - Reason: 12312
 - Status: Pending
 - Suggested on 2025-12-22 15:09
 - Review notes (optional)
 -
 -

(d) Admin Panel: Approve/Reject/Delete + Notes

Figure 8: UI Components: (a) Demand heatmap visualization, (b) Form with PostalCode validation, (c) User suggestion list, (d) Admin review panel

User vs Admin:

- **Users:** See pending + approved suggestions only
- **Admin:** Enter password → see all + approve/reject/delete

4. Integration of Explored Datasets

Datasets (Ladesäulenregister, residents, geodata) were explored in Part 1 and migrated into DDD structure:

```
core/methods.py -> src/shared/infrastructure/preprocessing/
                    src/demand/domain/events/
                    src/suggestion/domain/entities/
```

```
1 repo = SharedRepository()
2 service = SharedService(repo)
3 df_residents = service.get_processed_residents(path, df_geo, datasets_dir)
```

Listing 5: Method access via Dependency Injection

Data Formatting (handled in Infrastructure):

- Header detection: Auto-skips metadata rows in CSV
- PLZ normalization: `pd.to_numeric(PLZ, errors='coerce')`
- Coordinate format: German comma decimals → dots

Table 5: DDD Adherence Summary

DDD Principle	Implementation
Entities/VOs	PostalCode, ChargingSuggestion (self-validating)
Repositories	Interface in Domain, implementation in Infrastructure
Services	Thin orchestrators, no business logic
Exceptions	Domain-specific (InvalidSuggestionException)

Layer Dependencies:

```
Presentation → Application → Domain ← Infrastructure
```

(Infrastructure implements interfaces defined in Domain)

5. LLM Integration

LLMs were utilized throughout development to accelerate implementation (see Table 6).

Table 6: LLM Usage and Value Added

Use Case	Tool	Value Added
Architecture refactoring	Claude Code	Decomposed <code>core/methods.py</code> into DDD bounded contexts
Test generation	Claude Code	Edge-case tests for state machine transitions
Code optimization	GitHub Copilot	Vectorized Pandas operations for heatmap generation
Documentation	Claude Code	Consistent docstrings, README

```
1 # Prompt: "Generate tests for invalid state transitions"
2 # Result: Identified non-obvious cases like preventing duplicate reviews
3
4 def test_cannot_approve_already_approved(self, setup):
5     service, repo = setup
6     service.create_suggestion("10115", "Test", "Reason")
7     service.review_suggestion(1, status="approved", reviewer="Admin")
8
```



```

9   with pytest.raises(InvalidSuggestionException):
10      service.review_suggestion(1, status="approved", reviewer="Admin2")

```

Listing 6: LLM-generated edge case test for state machine

6. Technical Challenges

Table 7: Technical Challenges and Solutions

Challenge	Problem	Solution
Decoupling Legacy	Monolithic <code>core/methods.py</code> difficult to maintain/test	Incremental refactoring with smoke tests as safety net
Suggestion Lifecycle	Preventing invalid transitions (Approved → Rejected)	State Machine pattern in Charging-Suggestion entity
Test Performance	File-based JSON made tests slow	In-Memory Repository variant via DI

7. Project Completion

7.1 Milestones Achieved

- ✓ Demand heatmap visualization
- ✓ Suggestion submission with validation
- ✓ Admin review (approve/reject/delete)
- ✓ User visibility filtering
- ✓ DDD architecture implementation
- ✓ Test coverage ~80% for business logic

7.2 Future Improvements

Functional:

- Demand score normalization – convert raw ratio to meaningful scale (0-100) or categories
- Community voting on suggestions (User Story 4)
- Email notifications for status changes

DDD/Architecture:

- True Domain Events with Event Bus
- CQRS (separate Read/Write models)
- Repository interfaces as formal abstractions
- Unit of Work pattern

7.3 Technical Innovations & Lessons Learned

Innovations:

- **State Machine Pattern** – Centralized lifecycle logic in Domain Entity
- **In-Memory Repository** – Seamless test/production switching via DI
- **Clean Architecture** – Domain independent of external frameworks

Lessons Learned:

1. **TDD is not all-or-nothing** – Pragmatic adoption works well
2. **DDD pays off for complexity** – Bounded contexts made features easy
3. **LLMs accelerate, not replace** – Best for tests, refactoring, consistency
4. **Documentation alongside code** – Prevents technical debt